Технология разработки программ в среде Linux

В операционной системе Linux для компиляции программ, написанных на языке **C\C++,** могут быть использованы различные компиляторы. Основные из них - GNU C/C++ Compiler (gcc) и Clang. Исторически так сложилось, что gcc фактически является стандартом для Ubuntu. Для компиляции С-программы, состоящей из одного файла, используется компилятор **cc** или **gcc**, а для компиляции программы, написанной на **C++,** используется компилятор **g++.**

Исторически сложилось так, что, если задать строку компиляции ***cc proba.c***, то результатом будет исполняемый файл **a.out**. Это имя подставляется автоматически, если явно не указано имя выходного файла. Для задания имени выходного файла используется опция ***–o <имя файла>.*** При этом тип файла выбирается автоматически. Если перед именем компилируемого файла стоит ***–c***, то создается объектный выходной файл с расширением ***.o***, в противном случае создается исполняемый файл.

Вызов ***cc*** также используется при необходимости собрать исполняемый файл из нескольких объектных. Для этого необходимо вызвать его следующим образом:
cc <объектные модули> -o <имя исполняемого файла>

Если программа использует дополнительные библиотеки, возникает необходимость указания дополнительных путей поиска заголовочных и библиотечных файлов. Для этого используются опции ***–I<путь>*** и ***–L<путь>***. При необходимости могут присутствовать несколько опций ***–I*** и ***–L***.

Опция ***–g*** предназначена для включения в исполняемый файл отладочной информации. Это позволит привязать точки останова к соответствующим строкам файла с исходным текстом. Впоследствии отладочную информацию можно удалить из исполняемого файла с использованием утилиты ***strip***.

Вызов компилятора ***C++:***
g++ <объектные модули> -o <имя исполняемого файла>

Для отладки программ используется стандартный отладчик ***gdb***. Формат его вызова следующий:
gdb <исполняемый файл> [<core файл>|<pid процесса>]
core файл – файл, создаваемый при аварийном выходе программы при возникновении критической ошибки. Если ***core файл*** указан, он будет автоматически разобран отладчиком ***gdb,*** и можно будет определить, в какой точке программы произошла ошибка. Если указан ***pid процесса***, отладчик подключится к этому процессу.

Рассмотрим команды отладчика ***gdb***.
b <номер строки>

или

b <имя исходного модуля>:<номер строки> -устанавливает точку останова в заданной строке в указанном или текущем модуле;

r - запускает программу на выполнение;

n - выполняет очередную строку программы без входа в функцию;

s - выполняет очередную строку программы с входом в функцию;

p <имя переменной> - выводит содержимое переменной;

q - выход из отладчика.

Пример

В текстовом редакторе (например, ***vim,*** вызов: ***vim hello.c)*** написать программу ***hello.c,*** выводящую на экран фразу ***"HELLO Linux".*** Скомпилировать полученную программу компилятором ***gcc: gcc hello.c -o hello.***

Запустить полученный файл ***hello*** на выполнение:

./hello

Пример Hello, World!

Файл ***main.c:***

```
#include <stdio.h>

int main (void)
{
  printf ("Hello, World!\n");
  getchar ();
  return 0;
}
```

Устанавливаем необходимые зависимости:

apt update && apt install gcc

компилируем данную программу

cc main.c -o main

запускаем её от имени обычного пользователя:

./main